

Basic (a simple version log) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below use a stand-in for Doc13's real test suite, `run_eval_suite(prompt_text)`, so the script runs on its own. In your real Project 5 code, swap it for your actual Doc13 suite import.

Basic Version

Approach 1 — the direct way

```
import json
from datetime import datetime

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

versions = []
versions.append({
    "version_name": "v1",
    "prompt_text": "Answer the customer's question.",
    "created_at": str(datetime.now()),
})
versions.append({
    "version_name": "v2",
    "prompt_text": "Answer the customer's question politely and cite the source.",
    "created_at": str(datetime.now()),
})

with open("versions.json", "w") as f:
    json.dump(versions, f, indent=2)

for version in versions:
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])
```

Expected output:

```
v1 0.6  
v2 1.0
```

This version works correctly for what the exercise asks. It rebuilds `versions.json` from scratch every run (no loading of prior entries first) and uses `str(datetime.now())` for a quick, locally-formatted timestamp — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — reusable save/load functions

```
import json
from datetime import datetime, timezone
from pathlib import Path

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_version(name: str, prompt_text: str, path: str) -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

save_version("v1", "Answer the customer's question.", "versions.json")
save_version("v2", "Answer the customer's question politely and cite the source."

for version in load_versions("versions.json"):
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])
```

Expected output:

```
v1 0.6
```

```
v2 1.0
```

Approach 2 — same idea, printed as a readable table

```
def print_score_table(versions: list[dict]) -> None:
    print(f"{'version':<10}{'score':<8}{'created_at'}")
    for version in versions:
        result = run_eval_suite(version["prompt_text"])
        print(f"{version['version_name']:<10}{result['score']:<8}{version['create

print_score_table(load_versions("versions.json"))
```

Expected output:

version	score	created_at
v1	0.6	2026-09-11T14:03:00.123456+00:00
v2	1.0	2026-09-11T14:03:00.234567+00:00

Difference from Basic: both approaches add full type hints and split the work into `save_version` / `load_versions` functions that *load before they save*, so calling `save_version` twice in a row (once for v1, once for v2) builds up one growing list in `versions.json` instead of the Basic Version's approach of building the whole list in memory first and writing it once. The timestamp also switches to `datetime.now(timezone.utc).isoformat()` — a sortable, unambiguous string, instead of `str(datetime.now())`'s locale-dependent format. Approach 2 is a pure formatting improvement — same data, read as a table instead of separate print lines.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — append-only, with a duplicate-prompt warning

```
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_version(name: str, prompt_text: str, path: str) -> dict:
    versions = load_versions(path)
    prompt_hash = hashlib.sha256(prompt_text.encode("utf-8")).hexdigest()

    for existing in versions:
        if existing.get("prompt_hash") == prompt_hash:
            print(f"Warning: prompt text for '{name}' matches existing version '{existing.get('version_name')}'")

    entry = {
        "version_name": name,
        "prompt_text": prompt_text,
        "prompt_hash": prompt_hash,
        "created_at": datetime.now(timezone.utc).isoformat(),
    }
    versions.append(entry)
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)
    return entry

save_version("v1", "Answer the customer's question.", "versions.json")
save_version("v2", "Answer the customer's question politely and cite the source.")
save_version("v3", "Answer the customer's question.", "versions.json") # same text
```

Expected output:

Warning: prompt text for 'v3' matches existing version 'v1'

`versions.json` still gets all 3 entries — the warning tells you about the duplicate, it doesn't block the save. Nothing in this log is ever overwritten or deleted; every call to `save_version` only ever appends.

Approach 2 — SQLite instead of a flat JSON file

```
import sqlite3
from datetime import datetime, timezone

def init_db(path: str) -> None:
    conn = sqlite3.connect(path)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS versions (
            version_name TEXT,
            prompt_text TEXT,
            created_at TEXT
        )
    """)
    conn.commit()
    conn.close()

def save_version(name: str, prompt_text: str, db_path: str) -> None:
    conn = sqlite3.connect(db_path)
    conn.execute(
        "INSERT INTO versions (version_name, prompt_text, created_at) VALUES (?, "
        "(name, prompt_text, datetime.now(timezone.utc).isoformat()), "
    )
    conn.commit()
    conn.close()

def load_versions(db_path: str) -> list[dict]:
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    rows = conn.execute("SELECT * FROM versions").fetchall()
    conn.close()
    result = []
    for row in rows:
        result.append(dict(row))
    return result

init_db("versions.db")
save_version("v1", "Answer the customer's question.", "versions.db")
save_version("v2", "Answer the customer's question politely and cite the source."
```

```
for version in load_versions("versions.db"):
    print(version["version_name"], version["created_at"])
```

Expected output:

```
v1 2026-09-11T14:03:00.123456+00:00
v2 2026-09-11T14:03:00.234567+00:00
```

`INSERT` only ever adds a row — there's no equivalent of "open the file in write mode and forget to load first," which is the exact mistake a flat-file version could make by accident. SQLite also handles two processes writing to `versions.db` around the same time safely, which a plain JSON file, read-then-rewritten-whole, does not.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `save_version` already loads-then-appends, so it doesn't lose history — but it has no way to notice an accidental duplicate, and a flat JSON file being read and rewritten whole is still not safe if two processes write at the same moment. Approach 1 adds the duplicate-prompt hash check directly on top of Intermediate's JSON approach — a small, dependency-free addition. Approach 2 swaps the storage layer entirely for SQLite, where every write is a single `INSERT` statement — safer under concurrent writers, and closer to what the Build Task's `versioning.py` should actually use once this log needs to survive being read by more than one script at a time.

Which one should you actually write? For this exercise as written — two versions of one prompt, run once — Intermediate Approach 1 is already enough. Reach for Advanced Approach 1's hash check once you're worried about accidentally re-saving the same prompt under a new name (easy to do after a few dozen versions). Reach for Advanced Approach 2's SQLite version for the Build Task itself — it's what `versioning.py` should build on, since the release gate and rollback function both need to read this log reliably from a separate script run.